MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Cuibari Vladislav FAF-243

Report

Laboratory Work No. 3

Operating Systems

Topic: The Command Line: Shell, Pipes and Text Filters

Checked by:
Reițman Patricia, *university assistant*

Chișinău – 2026

Contents

INTRODUCTION 4
1 MOVING FAST AND FINDING HELP 5
1.1 Tab Completion and Command History 5
1.2 Built-in and External Commands 6
1.3 The PATH Variable 6
1.4 Exit Codes 7
1.5 Finding Help Without a Browser 7
1.6 Answers to the Observe Questions 8
2 WHAT THE SHELL DOES TO YOUR LINE 9
2.1 Preparing the Test Directory 9
2.2 Globbing 9
2.3 Spaces in File Names 10
2.4 Quoting 10
2.5 Brace Expansion, Arithmetic and Command Substitution 10
2.6 Predictions and Real Output 11
2.7 Answers to the Observe Questions 11
3 REDIRECTION, CHAINING AND JOBS 13
3.1 Separate Files for stdout and stderr 13
3.2 The Order of Redirections 13
3.3 The Operators ; && || 14
3.4 Background Jobs 14
3.5 Append and Input Redirection 15
3.6 Answers to the Observe Questions 15
4 ONE-LINERS OVER THE ACCESS LOG 17
4.1 Q1. How many requests failed? 17
4.2 Q2. How many different pages were requested, and which? 17
4.3 Q3. How many requests did each page get? 18
4.4 Q4. Which user or users have the most failed requests? 18
4.5 Q5. How many requests did user3 make, and how many of them failed? 18
4.6 Q6. The last 3 failed requests, only time and user 19
4.7 Q7. Login shells in /etc/passwd 19
4.8 Q8. ls /etc | wc -l versus ls -l /etc | wc -l 20
4.9 The Complete oneliners.sh 20
4.10 Bonus: Failed Requests Whose Minute Ends in 0 21
5 YOUR FIRST SCRIPT 23
5.1 Contents of report.sh 23

5.2 Making the Script Executable and Running It 23
5.3 Error Handling..... 24
5.4 Answers to the Observe Questions..... 24
CONCLUSION 26
REFERENCES..... 27

INTRODUCTION

This laboratory work is about the command line, the most direct way of talking to a Unix-like operating system. Laboratory Work No. 4 looks at the tiny shell of xv6; here the shell of a real Linux system, bash, is used from the outside. The work covers fast editing with Tab completion and history, what the shell does to a command line before the command runs (globbing, quoting, expansions), the redirection of the standard streams, chaining of commands and background jobs, and finally answering questions about data with one-line pipelines of small text filters, followed by a first shell script.

The purpose of the work is to move fast in bash, to predict how the shell rewrites a line before it is executed, to redirect output where it is needed, and to answer real questions about a data set with small filters joined by pipes.

The report follows the five parts of the handout. Every exercise shows the commands, a terminal figure and a comment. The predictions of Part 2, the answers to the Observe boxes and the answers to questions Q1–Q8 are given in separate subsections (1.6, 2.6, 2.7, 3.6, 4.1–4.8 and 5.5). The first number of every figure is the number of the part, the second is the number of the figure inside that part.

All commands were executed in bash on Ubuntu 26.04.1 LTS, running in an Oracle VirtualBox virtual machine, under the user account `dalvgame`; the working directory of the lab is `/home/dalvgame/os-lab3`. The deliverables `oneliners.sh` and `report.sh` are stored in the `lab3/` folder of the course repository together with this report.

The data set of Part 4, an artificial web access log of 500 requests, was generated with the block given in the handout:

```
mkdir -p ~/os-lab3 && cd ~/os-lab3
for i in $(seq 1 500); do
echo "2026-10-05 10:$(printf %02d $((i % 60))) user$((i % 7)) $([ $((i % 5)) -eq
0 ] && echo FAIL || echo OK) /page$((i % 4))"
done > access.log
head -3 access.log ; wc -l access.log
```

The check passed: the last command printed the required `500 access.log`. Every line has five fields separated by single spaces: date, time, user (`user0` … `user6`), result (`OK` or `FAIL`) and page (`/page0` … `/page3`). The output of the check was:

```
2026-10-05 10:01 user1 OK /page1
2026-10-05 10:02 user2 OK /page2
2026-10-05 10:03 user3 OK /page3
500 access.log
```

1 MOVING FAST AND FINDING HELP

This chapter corresponds to Part 1 of the handout. The exercises on completion, history, command lookup, PATH, exit codes and built-in help were done one by one in a terminal.

1.1 Tab Completion and Command History

```
ls /usr/sh<Tab><Tab>
cd /et<Tab>
# three commands, then Ctrl+R and a few letters of the first one
```

The evidence for this exercise is shown in Figure 1.1/1.2.

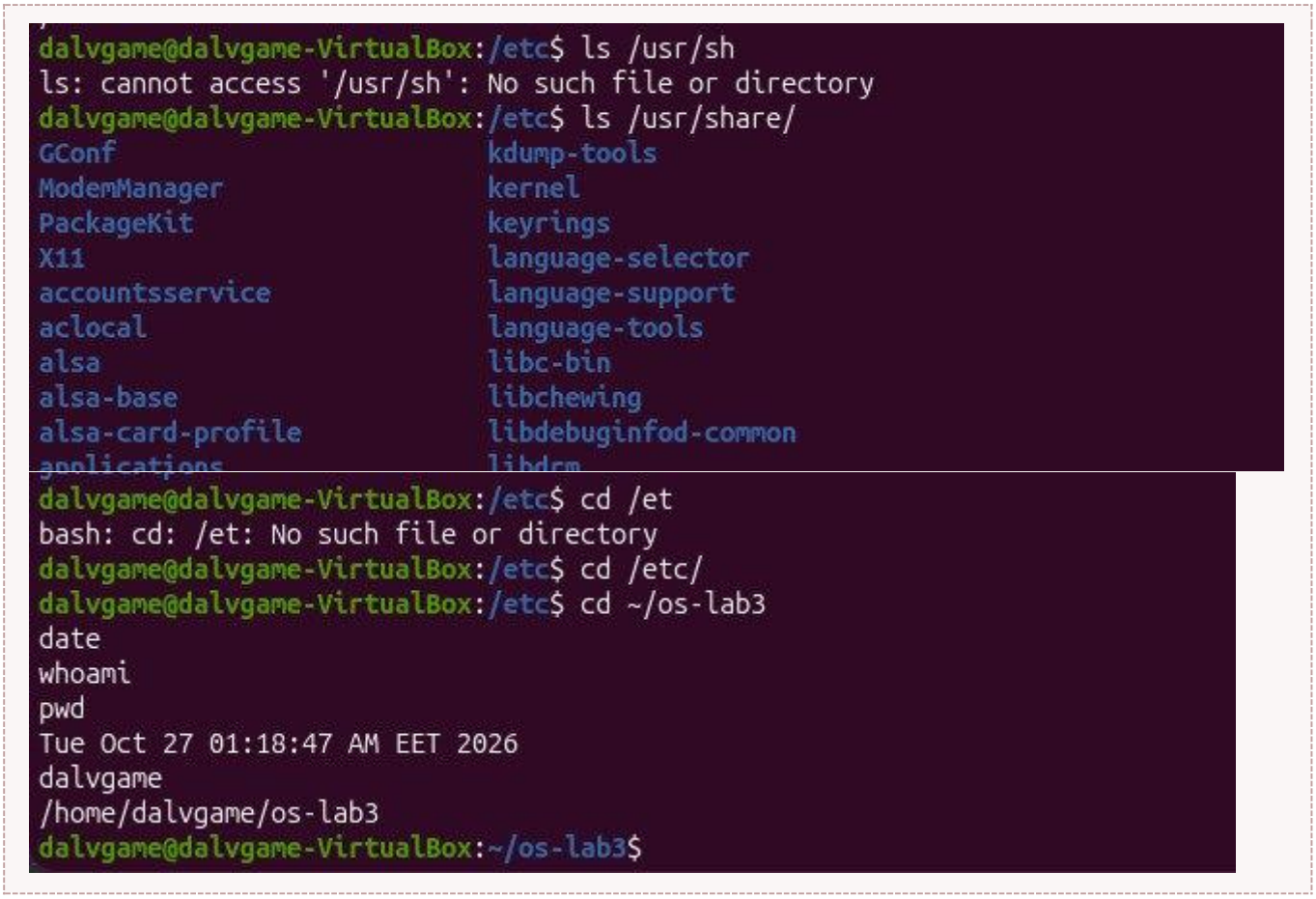


Figure 1.1/1.2 – Tab completion

Comment and observation

`/usr/sh` does not exist as a name, but Tab completes a prefix: it was extended to `/usr/share/`, the only entry that starts with `sh`. A second Tab changes nothing when there is just one candidate; with a shorter prefix that matches several names, two presses of Tab print the list of candidates and leave the typed text unchanged. `cd /et` followed by Tab was completed to `cd /etc/`.

For the history exercise three commands were run (`date`, `whoami`, `pwd`). Ctrl+R switched the prompt to `(reverse-i-search)`; after a few letters of the first command bash showed the most recent command containing them, and Enter executed it again. Tab therefore saves typing

and avoids typing errors, and Ctrl+R finds an old command by any fragment instead of retyping it.

1.2 Built-in and External Commands

```
type cd ; type ls ; type echo ; which ls
```

The evidence for this exercise is shown in Figure 1.3.



Figure 1.3 – Output of type and which

Comment and observation

`type` reports `cd` and `echo` as shell builtins. `ls` is not a builtin: copy what your type ls printed – on Ubuntu it is usually "ls is aliased to `ls --color=auto`"; the program behind it is the file /usr/bin/ls. `which ls` finds the file by searching the directories of PATH. A builtin is executed by the shell process itself, whereas an external command needs a new process that loads a file from disk.

1.3 The PATH Variable

```
echo $PATH
echo $PATH | tr ':' '\n'
echo $PATH | tr ':' '\n' | wc -l
```

The evidence for this exercise is shown in Figure 1.4.



Figure 1.4 – The directories in PATH

Comment and observation

PATH is a colon-separated list of directories. When a command name contains no slash, the shell looks for an executable file of that name in these directories from left to right and runs the first one it finds. `tr` replaced every colon by a newline, which shows the list one directory per line; `wc -l` counted 9 directories. `ls` is found in `/usr/bin`.

1.4 Exit Codes

```
nosuchprog ; echo "exit code: $?"
```

The evidence for this exercise is shown in Figure 1.5.

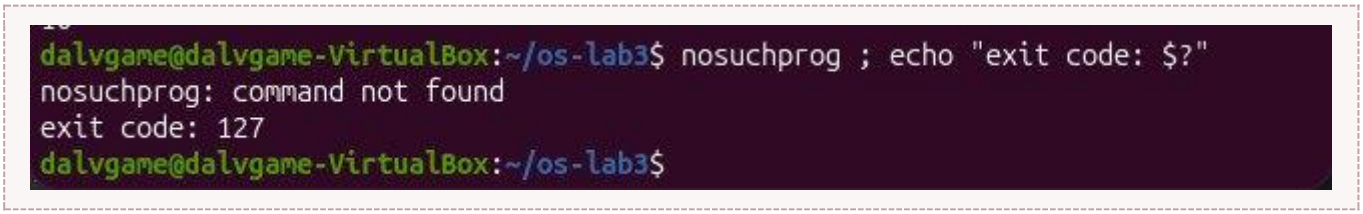


Figure 1.5 – Exit code of a missing command

Comment and observation

`$?` holds the exit code of the last command. A command that cannot be found returns 127, and the shell prints `command not found`. By convention 0 means success and every non-zero value is some kind of failure; the operators `&&` and `||` of Part 3 are built on this convention.

1.5 Finding Help Without a Browser

```
man ls                      # inside: /sort, n for next, q to quit
ls --help | head
man -k "list directory"
help cd
```

The evidence for this exercise is shown in Figure 1.6.

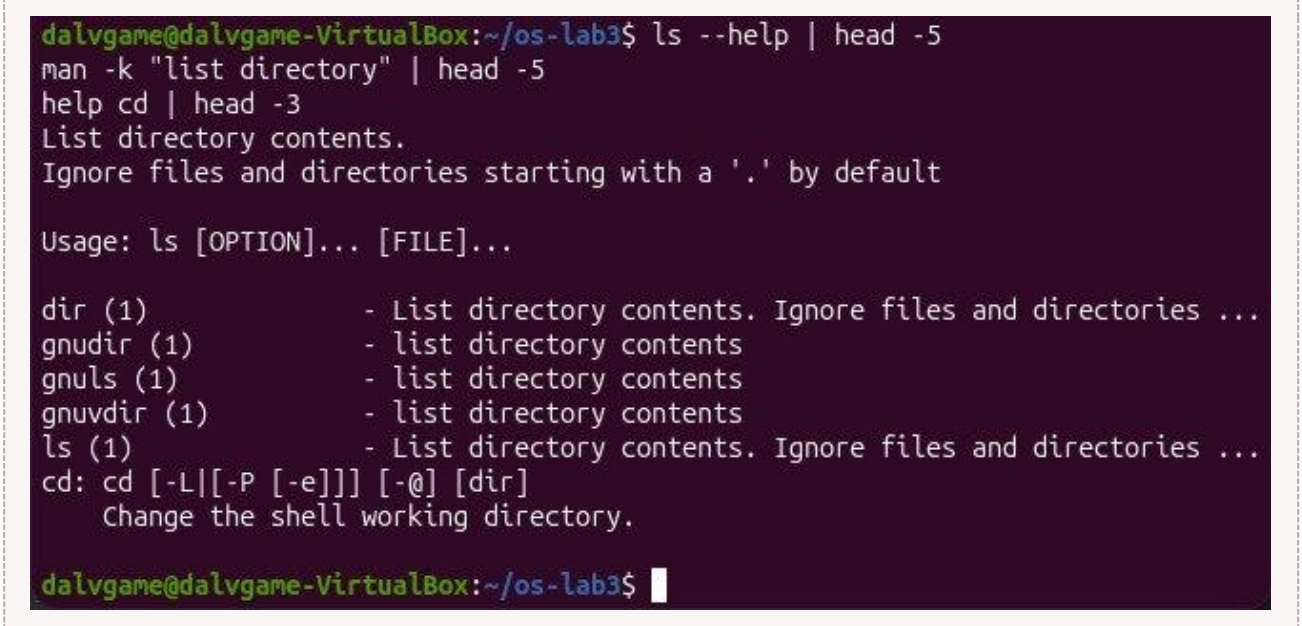


Figure 1.6 – Help from the manual and from the command itself

Comment and observation

Four sources of help were used. `man ls` shows the full manual page: `/sort` searches for a word, `n` jumps to the next match and `q` quits. External programs describe themselves with `--help`. `man -k` searches the one-line descriptions of all manual pages for a keyword. The builtin `cd` has no page of its own and is described by the builtin `help`. In the manual of `ls` the option that sorts by modification time is `-t` (sort by time, newest first).

1.6 Answers to the Observe Questions

(1) Built-ins. `cd` and `echo` are built-ins; `ls` is an external program (`/usr/bin/ls`). `cd` MUST be a built-in because the current working directory is a property of a process. An external program runs in a child process, so a `cd` executed there would change the directory of the child only, and the child disappears when it finishes; the directory of the shell that started it would stay the same. Only the shell itself can change its own working directory.

(2) PATH. The PATH contains 9 directories (Figure 1.3), and `ls` is in `/usr/bin`.

(3) Exit code. A command that is not found returns 127 (Figure 1.4).

(4) Sorting by time. The option is `-t` (found in `man ls`).

2 WHAT THE SHELL DOES TO YOUR LINE

This chapter corresponds to Part 2 of the handout. For every line the prediction was written down first and then the line was executed; the comparison is in Table 2.1 in Section 2.6.

2.1 Preparing the Test Directory

```
mkdir -p ~/os-lab3/play && cd ~/os-lab3/play
touch a.c b.c file1.txt file2.txt notes.txt "alfa beta.txt"
ls
```

The evidence for this exercise is shown in Figure 2.1.



Figure 2.1 – Files created in the play directory

Comment and observation

The directory contains six files: `a.c`, `b.c`, `file1.txt`, `file2.txt`, `notes.txt` and one whose name contains a space, `alfa beta.txt`. Because the output goes to a terminal, `ls` shows the name with the space inside single quotes.

2.2 Globbing

```
echo *.c
echo file?.txt
echo [a-b]*
echo *.xyz
```

The evidence for this exercise is shown in Figure 2.2.

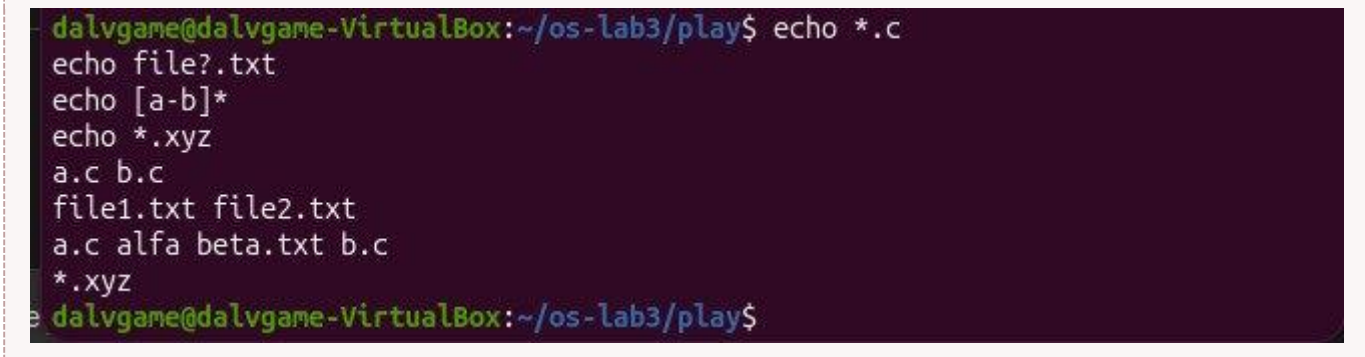


Figure 2.2 – Globbing results

Comment and observation

`*.c` was replaced by the two names that end in `.c`; `file?.txt` matched exactly one character in place of `?` (`file1.txt`, `file2.txt`); `[a-b]*` matched every name that begins with `a` or

`b`, so besides `a.c` and `b.c` it also produced `alfa beta.txt`. The pattern `*.xyz` matches nothing, and in that case bash leaves the pattern unchanged, so `echo` printed the text `*.xyz`.

2.3 Spaces in File Names

```
ls alfa beta.txt
ls "alfa beta.txt"
ls alfa\ beta.txt
```

The evidence for this exercise is shown in Figure 2.3.



Figure 2.3 – The ls error on a name with a space, and the fix

Comment and observation

The first line failed with two errors, `cannot access 'alfa'` and `cannot access 'beta.txt'`: the shell split the line at the space into two arguments, and `ls` looked for two files that do not exist. A name in double quotes, or with the space escaped by a backslash, is a single argument, and `ls` prints `'alfa beta.txt'`.

2.4 Quoting

```
echo '$HOME' "$HOME" \$HOME
```

The evidence for this exercise is shown in Figure 2.4.

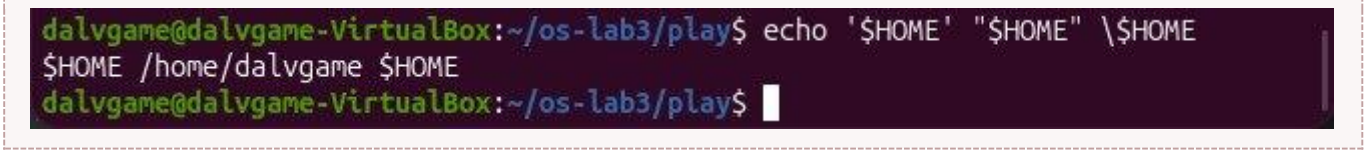


Figure 2.4 – Single quotes, double quotes and backslash

Comment and observation

The three forms gave different results: inside single quotes `$HOME` stayed literal, inside double quotes it was replaced by `/home/dalvgame`, and with a backslash before the dollar sign the shell treated it as an ordinary character and printed `$HOME`.

2.5 Brace Expansion, Arithmetic and Command Substitution

```
echo {1..5} ; echo $((7 * 6))
echo "today is $(date +%A), I am $(whoami)"
touch log{1..3}.txt && ls log*
```



Figure 2.5 – Brace expansion, arithmetic and command substitution

Comment and observation

`{1..5}` was expanded to the five words `1 2 3 4 5`, and `$((7 * 6))` was replaced by `42`. In the second line the shell first ran `date +%A` and `whoami`, put their output into the line and only then started `echo`. In the third line `log{1..3}.txt` became `log1.txt log2.txt log3.txt`, so one `touch` created three files, which the pattern `log*` then listed.

2.6 Predictions and Real Output

Table 2.1 – Predictions compared with the real output

Command	My prediction	Real output	Explanation (if different)
`echo *.c`	Not recorded	a.c b.c	–
`echo file?.txt`	Not recorded	file1.txt file2.txt	–
`echo [a-b]*`	Not recorded	a.c alfa beta.txt b.c	The range matches the first letter of every name, so `alfa beta.txt` is included too.
`echo *.xyz`	Not recorded	*.xyz	No file matches, so bash keeps the pattern and `echo` receives it as an ordinary word.
`ls alfa beta.txt`	Not recorded	two errors: cannot access 'alfa' and 'beta.txt'	Word splitting: the shell passes two arguments, not one.
`ls "alfa beta.txt"`	Not recorded	'alfa beta.txt'	Same file; `ls` adds the quotes itself because the name contains a space.
`echo '$HOME' "$HOME" \$HOME`	Not recorded	$HOME /home/dalvgame $HOME	–
`echo {1..5} ; echo $((7 * 6))`	Not recorded	1 2 3 4 5 and 42	–
`echo "today is $(date +%A), I am $(whoami)"`	Not recorded	today is Monday, I am dalvgame	– (the weekday comes from the clock of the machine)
`touch log{1..3}.txt && ls log*`	Not recorded	log1.txt log2.txt log3.txt	– (`ls` puts the names in columns on a terminal)

2.7 Answers to the Observe Questions

(1) Predictions. The table in Section 2.6 gives, for every line, the prediction, the real output and an explanation where they differ.

(2) Who produced the file names? The shell. `echo` knows nothing about files: before starting it, bash expands the pattern `*.c` into the list of matching names (globbing) and passes `a.c b.c` to `echo` as two ordinary arguments. `echo` simply prints its arguments.

(3) Why did ls alfa beta.txt fail? The shell splits the line into words at spaces, so `ls` received two arguments, `alfa` and `beta.txt`, and neither file exists. Two fixes: quote the name, `ls "alfa beta.txt"`, or escape the space with a backslash, `ls alfa\ beta.txt` (single quotes work as well).

(4) Single versus double quotes. Single quotes keep everything literal: no variable, command, arithmetic or glob expansion takes place inside them. Double quotes still protect spaces and wildcards from splitting and globbing, but allow `$variable`, `$(command)`, `$((expression))` and backslash escapes to be expanded. A backslash outside quotes removes the special meaning of the one character that follows it.

Why does ls show 'alfa beta.txt' in quotes? The name contains a character that is special for the shell. When its output goes to a terminal, GNU `ls` prints such names in shell-quoted form, so that the text can be copied back into a command and still mean the same file. When the output goes into a pipe or a file, the name is printed without quotes.

3 REDIRECTION, CHAINING AND JOBS

This chapter corresponds to Part 3 of the handout. The commands were run in `~/os-lab3`, where `access.log` is stored.

3.1 Separate Files for stdout and stderr

```
ls /etc /nope > out.txt 2> err.txt
wc -l out.txt ; cat err.txt
```

The evidence for this exercise is shown in Figure 3.1.

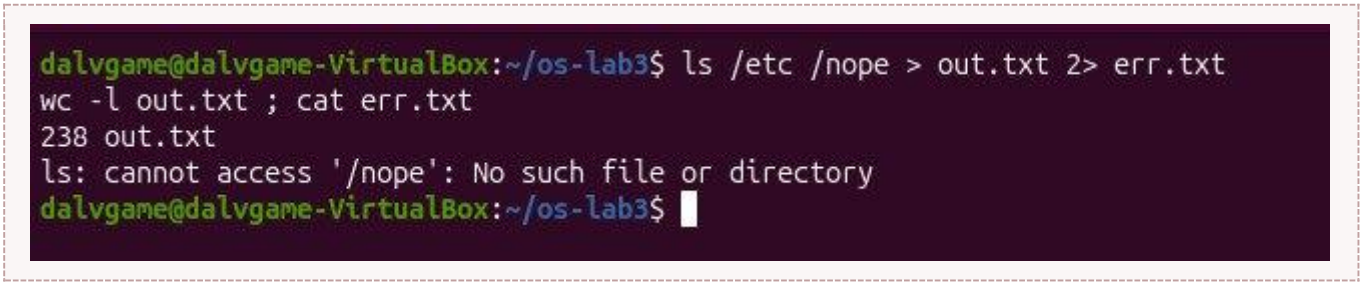


Figure 3.1 – stdout and stderr in separate files

Comment and observation

The first command printed nothing on the screen: its standard output (descriptor 1) went to `out.txt` and its standard error (descriptor 2) to `err.txt`. `out.txt` holds 238 lines (the heading `/etc:` followed by the names in the directory), and `err.txt` contains the single message `ls: cannot access '/nope': No such file or directory`.

3.2 The Order of Redirections

```
ls /etc /nope > all.txt 2>&1 ; tail -2 all.txt ; grep -c nope all.txt
ls /etc /nope 2>&1 > all.txt ; tail -2 all.txt ; grep -c nope all.txt
```

The evidence for this exercise is shown in Figure 3.2.



Figure 3.2 – Comparing the order of redirections

Comment and observation

With `> all.txt 2>&1` nothing appeared on the screen, and `grep -c nope all.txt` found the error message in the file (count 1): both streams ended up in `all.txt`. With `2>&1 > all.txt` the error message appeared on the screen and the file contained only the listing (count 0). The same two redirections in the opposite order therefore give different results.

3.3 The Operators ; && ||

```
ls /nope 2> /dev/null || echo "failed with code $?"
mkdir -p tmp && cd tmp && echo "I am in $(pwd)" ; cd ~/os-lab3
```

The evidence for this exercise is shown in Figure 3.3.

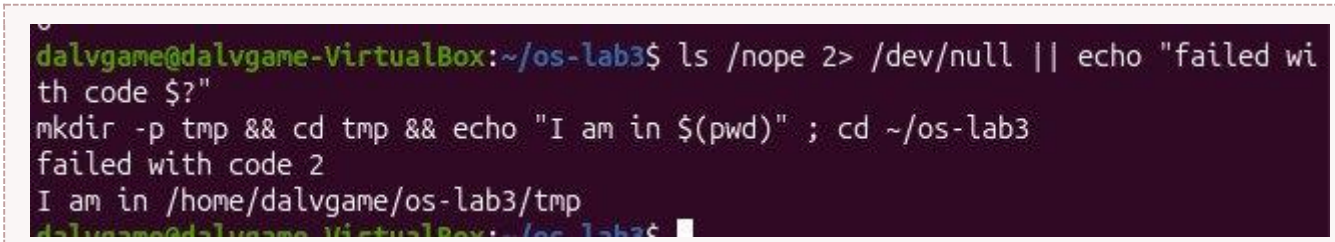


Figure 3.3 – The ; && || operators

Comment and observation

The error message of `ls /nope` was thrown away into `/dev/null`, and because `ls` failed, the part after `||` ran and reported the exit code 2. In the second line `mkdir`, `cd` and `echo` ran one after another, each only after the previous one succeeded; `echo` showed the new directory `/home/dalvgame/os-lab3/tmp`. The final `cd ~/os-lab3` after `;` runs in any case and returned to the lab directory.

3.4 Background Jobs

```
sleep 100 &
jobs
kill %1
jobs
```

The evidence for this exercise is shown in Figure 3.4.

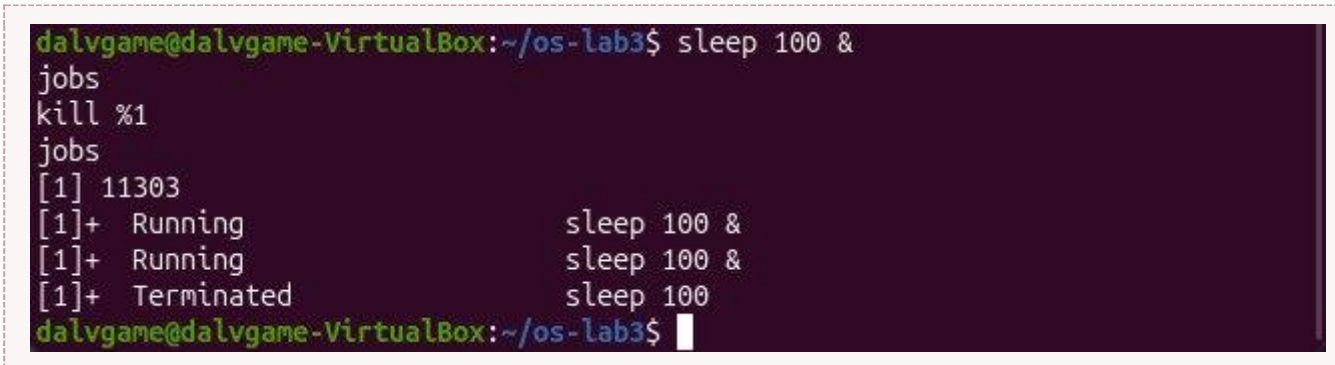


Figure 3.4 – Running a background job and stopping it

Comment and observation

The `&` at the end of the line started `sleep 100` in the background; the shell printed the job number and the process number and gave the prompt back at once. `jobs` listed the job as `Running`. `kill %1` sent the default termination signal to job number 1, and the next `jobs` reported it as `Terminated`.

3.5 Append and Input Redirection

```
echo first > notes.md ; echo second >> notes.md ; cat notes.md
wc -l < access.log
wc -l access.log
```

```
dalvgame@dalvgame-VirtualBox:~/os-lab3$ echo first > notes.md ; echo second >> n
otes.md ; cat notes.md
wc -l < access.log
wc -l access.log
first
second
500
500 access.log
dalvgame@dalvgame-VirtualBox:~/os-lab3$
```

Figure 3.5 – Append (>>) and input redirection (<)

Comment and observation

> creates the file (or empties an existing one), >> appends to it: after the two `echo` commands `notes.md` holds the lines `first` and `second`. `wc -l < access.log` printed only `500`, while `wc -l access.log` printed `500 access.log`.

3.6 Answers to the Observe Questions

(1) Which file got what? In Figure 3.1 the listing went to `out.txt` and the error message `ls: cannot access '/nope'…` went to `err.txt`.

(2) > all.txt 2>&1 versus 2>&1 > all.txt. The shell processes redirections from left to right, and `2>&1` means "make descriptor 2 point to the place where descriptor 1 points now". In `> all.txt 2>&1`, descriptor 1 is first connected to the file and then descriptor 2 becomes a copy of it, so both streams go to `all.txt`. In `2>&1 > all.txt`, descriptor 2 is first made a copy of descriptor 1 while it still points at the terminal, and only afterwards descriptor 1 is moved to the file; the errors therefore stay on the screen and only the listing reaches the file (Figure 3.2: count 1 versus count 0).

(3) wc -l < access.log versus wc -l access.log. With < the shell opens the file and gives it to `wc` as standard input; `wc` sees only a stream of bytes and does not know any file name, so it prints just the number. With the file name as an argument `wc` opens the file itself, knows its name and prints it after the count.

(4) The operators. `;` runs the next command after the previous one has finished, whatever its result; `&&` runs the next command only if the previous one succeeded (exit code 0); `||` runs the next command only if the previous one failed (non-zero exit code).

4 ONE-LINERS OVER THE ACCESS LOG

This chapter corresponds to Part 4 of the handout. Every question was answered with one command line over `access.log`; the lines were saved in `oneliners.sh`, each preceded by the question as a comment. The fields of the log are separated by a space: 1 date, 2 time, 3 user, 4 result, 5 page.

4.1 Q1. How many requests failed?

```
grep -c FAIL access.log
100
```

The evidence for this exercise is shown in Figure 4.1.

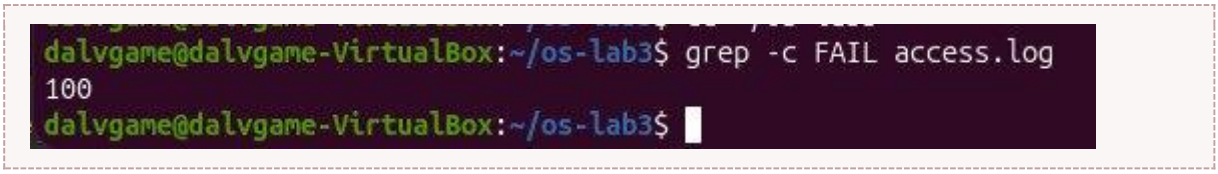


Figure 4.1 – Number of failed requests

Comment and observation

`grep -c` counts the matching lines instead of printing them. 100 requests failed, exactly every fifth request of the 500 (the generator writes `FAIL` when the counter is divisible by 5).

4.2 Q2. How many different pages were requested, and which?

```
cut -d' ' -f5 access.log | sort -u | wc -l ; cut -d' ' -f5 access.log | sort -u
4
/page0
/page1
/page2
/page3
```

The evidence for this exercise is shown in Figure 4.2.

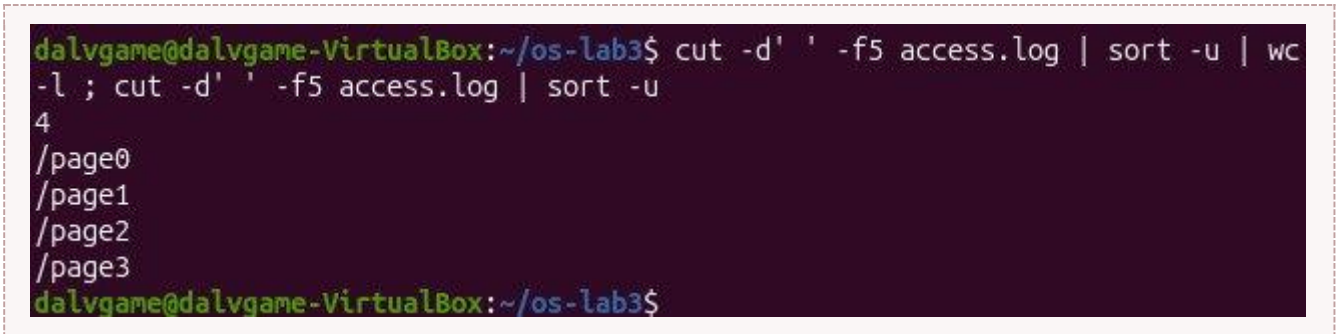


Figure 4.2 – Number and names of the requested pages

Comment and observation

`cut -d' ' -f5` keeps only the fifth field (the page), `sort -u` sorts it and removes duplicates, and `wc -l` counts what is left. There are 4 different pages: `/page0`, `/page1`, `/page2` and `/page3`.

4.3 Q3. How many requests did each page get?

```
cut -d' ' -f5 access.log | sort | uniq -c
    125 /page0
    125 /page1
    125 /page2
    125 /page3
```

The evidence for this exercise is shown in Figure 4.3.

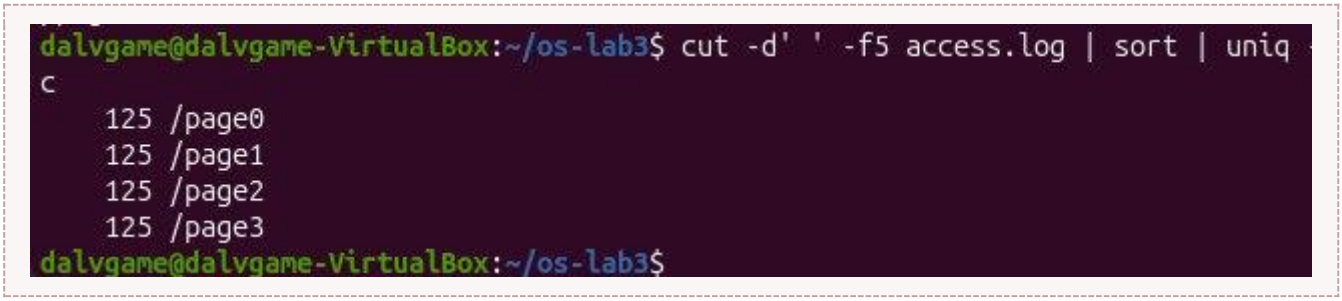


Figure 4.3 – Requests per page

Comment and observation

`uniq -c` counts runs of identical neighbouring lines, which is why `sort` has to come first. Every page received 125 requests. Self-check: 4 × 125 = 500, the number of lines of the log.

4.4 Q4. Which user or users have the most failed requests?

```
grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn | awk
'NR==1{m=$1} $1==m'
     15 user5
     15 user3
```

The evidence for this exercise is shown in Figure 4.4.

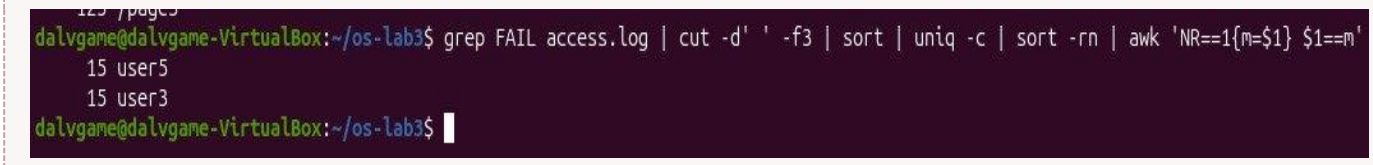


Figure 4.4 – The users with the most failed requests

Comment and observation

The pipeline counts the failed requests per user and sorts the counts in descending order. The maximum is shared, so `head -1` would hide one of the two users; the `awk` filter prints every line whose count equals the count of the first line. The answer is `user5` and `user3`, 15 failed requests each. Self-check: the other five users have 14 failures each (the per-user listing of Part 5 shows 14, 14, 14, 15, 14, 15, 14), and 15 + 15 + 5 × 14 = 100, which equals Q1.

4.5 Q5. How many requests did user3 make, and how many of them failed?

```
echo "total: $(grep -c ' user3 ' access.log) failed: $(grep ' user3 ' access.log
| grep -c FAIL)"
total: 72 failed: 15
```

The evidence for this exercise is shown in Figure 4.5.



Figure 4.5 – Requests and failures of user3

Comment and observation

The first `grep -c` counts all lines of `user3`; the second pipeline keeps only those lines that also contain `FAIL`. `user3` made 72 requests, and 15 of them failed. The spaces around `user3` in the pattern prevent a match inside a longer name.

4.6 Q6. The last 3 failed requests, only time and user

```
grep FAIL access.log | tail -3 | cut -d' ' -f2,3
10:10 user0
10:15 user5
10:20 user3
```

The evidence for this exercise is shown in Figure 4.6.



Figure 4.6 – The last three failed requests (time and user)

Comment and observation

`grep FAIL` selects the failed requests, `tail -3` keeps the last three of them and `cut -f2,3` prints only the time and the user: `10:10 user0`, `10:15 user5` and `10:20 user3`.

4.7 Q7. Login shells in /etc/passwd

```
cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
     40 /usr/sbin/nologin
      5 /bin/false
      2 /bin/bash
      1 /bin/sync
```

The evidence for this exercise is shown in Figure 4.7.

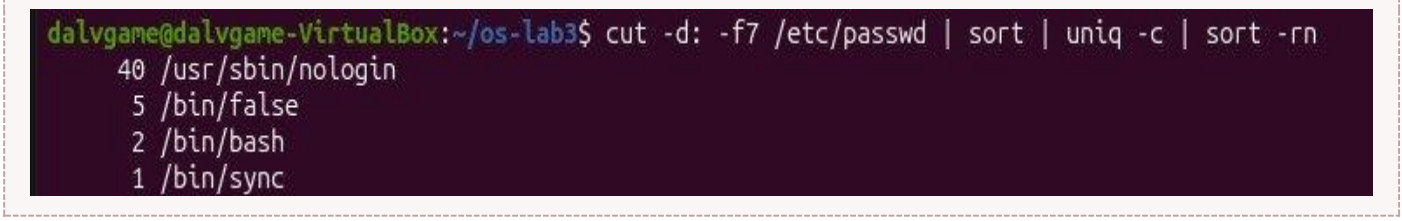


Figure 4.7 – Login shells and the number of accounts using each

Comment and observation

The seventh field of every line of `/etc/passwd` is the login shell. On this machine `/usr/sbin/nologin` is used by 40 accounts (system accounts that must not log in),

`/bin/false` by 5, `/bin/bash` by 2 and `/bin/sync` by 1. The numbers depend on the accounts of the machine where the command is run.

4.8 Q8. ls /etc | wc -l versus ls -l /etc | wc -l

```
ls /etc | wc -l ; ls -l /etc | wc -l
237
238
```

The evidence for this exercise is shown in Figure 4.8.

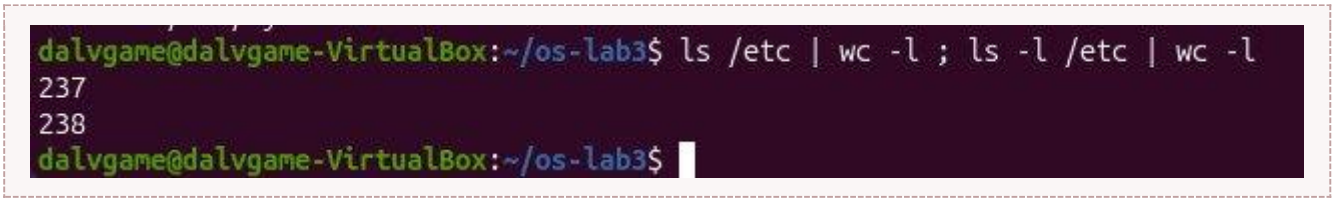


Figure 4.8 – ls /etc | wc -l versus ls -l /etc | wc -l

Comment and observation

The second number is larger by exactly one. `ls -l` starts its output with an extra summary line, `total N` (the number of disk blocks used by the listed files). That line is not a file, but `wc -l` counts lines and does not know the difference.

4.9 The Complete oneliners.sh

```
# Q1. How many requests failed?
grep -c FAIL access.log
# Q2. How many different pages were requested, and which?
cut -d' ' -f5 access.log | sort -u | wc -l ; cut -d' ' -f5 access.log | sort -u
# Q3. How many requests did each page get?
cut -d' ' -f5 access.log | sort | uniq -c
# Q4. Which user(s) have the most failed requests, and how many?
grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn | awk
'NR==1{m=$1} $1==m'
# Q5. How many requests did user3 make, and how many of them failed?
echo "total: $(grep -c ' user3 ' access.log) failed: $(grep ' user3 ' access.log
| grep -c FAIL)"
# Q6. Last 3 failed requests, only time and user
grep FAIL access.log | tail -3 | cut -d' ' -f2,3
# Q7. Login shells in /etc/passwd and how many accounts use each
cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
# Q8. Why do the two ls | wc -l differ by one?
ls /etc | wc -l ; ls -l /etc | wc -l
# Bonus. Minute ends in 0 and request failed
grep -cE ' 10:[0-5]0 user[0-9] FAIL ' access.log
```

The evidence for this exercise is shown in Figure 4.9.

```
dalvgame@dalvgame-VirtualBox:~/os-lab3$ bash oneliners.sh
100
4
/page0
/page1
/page2
/page3
    125 /page0
    125 /page1
    125 /page2
    125 /page3
     15 user5
     15 user3
total: 72 failed: 15
10:10 user0
10:15 user5
10:20 user3
     40 /usr/sbin/nologin
      5 /bin/false
      2 /bin/bash
      1 /bin/sync
237
238
50
dalvgame@dalvgame-VirtualBox:~/os-lab3$
```

Figure 4.9 – Full run of oneliners.sh

Comment and observation

The file runs with `bash oneliners.sh` inside `~/os-lab3`. All self-checks hold: the page counts of Q3 add up to 500 and the per-user failures behind Q4 add up to the 100 of Q1.

4.10 Bonus: Failed Requests Whose Minute Ends in 0

```
grep -cE ' 10:[0-5]0 user[0-9] FAIL ' access.log
50
```

```
dalvgame@dalvgame-VirtualBox:~/os-lab3$ grep -cE ' 10:[0-5]0 user[0-9] FAIL ' access.log
50
dalvgame@dalvgame-VirtualBox:~/os-lab3$
```

Figure 4.10 – Failed requests whose minute ends in 0

Comment and observation

The extended regular expression `10:[0-5]0` matches the minutes 00, 10, 20, 30, 40 and 50, and `FAIL` keeps only failed requests. The answer is 50 lines. This is expected: the minute is the counter modulo 60, so it ends in 0 exactly when the counter is a multiple of 10, and every

multiple of 10 is also a multiple of 5, that is, a failed request. Between 1 and 500 there are 50 multiples of 10.

5 YOUR FIRST SCRIPT

This chapter corresponds to Part 5 of the handout. The best one-liners were turned into a program, `report.sh`.

5.1 Contents of report.sh

```
#!/bin/bash
# usage: ./report.sh access.log
log=$1
if [ ! -f "$log" ]; then
echo "no such file: $log" >&2
exit 1
fi
total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")
echo "$total requests, $fails failed"
for n in 0 1 2 3 4 5 6; do
echo "user$n: $(grep -c "user$n FAIL" "$log") failures"
done
```

The evidence for this exercise is shown in Figure 5.1.

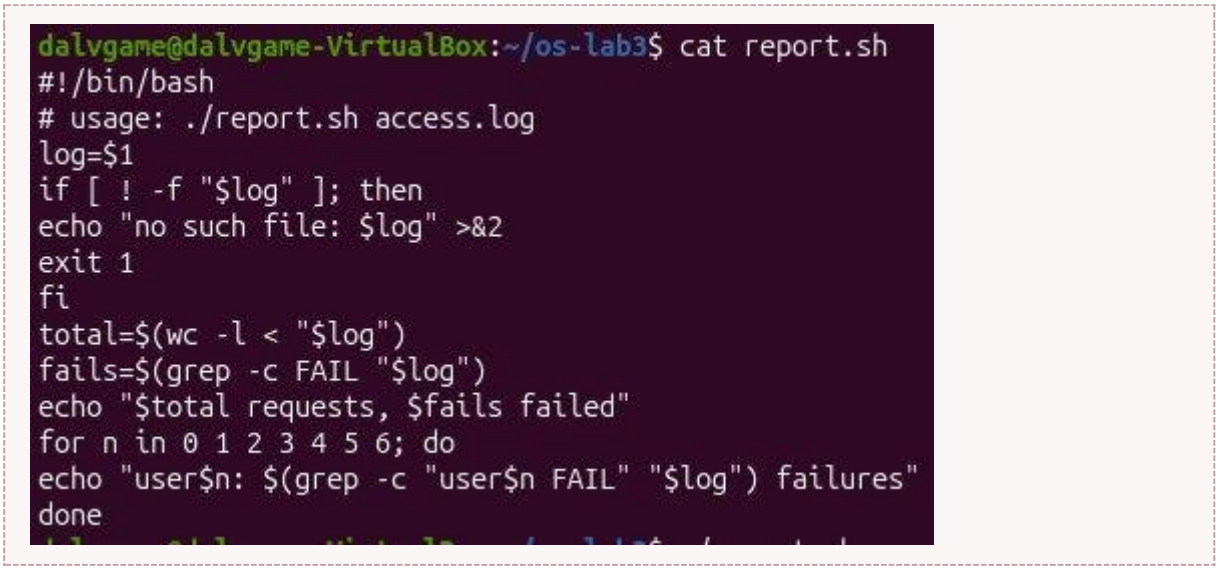


Figure 5.1 – Contents of report.sh

Comment and observation

The TODO of the handout was completed with a `for` loop over the numbers 0 to 6. For every number the loop counts the lines of the log that contain `userN FAIL` (the user, a space and the word FAIL, so that `user1` cannot match `user10`) and prints the result in the form `userN: X failures`.

5.2 Making the Script Executable and Running It

```
./report.sh access.log
bash: ./report.sh: Permission denied
chmod +x report.sh
./report.sh access.log
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
```

```
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
```

The evidence for this exercise is shown in Figure 5.2.

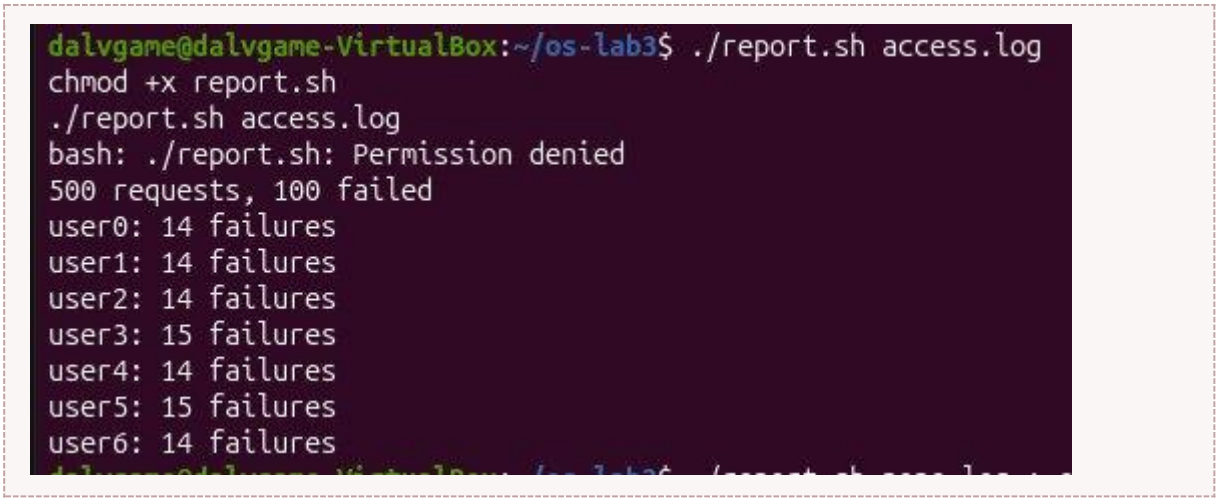


Figure 5.2 – Running the script on access.log

Comment and observation

A new file is created without the execute permission, so the first run is refused with `Permission denied`. After `chmod +x report.sh` the script runs and prints `500 requests, 100 failed` and the failures of every user. The numbers agree with Part 4: 14 + 14 + 14 + 15 + 14 + 15 + 14 = 100, and `user3` and `user5` are the two users with 15 failures.

5.3 Error Handling

```
./report.sh nope.log ; echo "exit code: $?"
no such file: nope.log
exit code: 1
```

The evidence for this exercise is shown in Figure 5.3.



Figure 5.3 – Error handling for a missing file

Comment and observation

For a file that does not exist the test `[ ! -f "$log" ]` is true, the script writes `no such file: nope.log` and ends with the exit code 1.

5.4 Answers to the Observe Questions

(1) What does #!/bin/bash do, and what happens before chmod +x? The first two characters `#!` (shebang) tell the kernel which interpreter must run the file: when the file is started, the

kernel starts `/bin/bash` and passes the script to it. Before `chmod +x` the file has no execute permission, so `./report.sh` fails with `Permission denied`.

(2) Why ./report.sh and not just report.sh? A command name without a slash is searched only in the directories listed in PATH, and the current directory is not one of them (Figure 1.3). `./report.sh` contains a slash, so the shell skips the search and runs exactly that file in the current directory. This is also a safety measure: a file with the name of a normal command, planted in the current folder, is not run by mistake.

(3) Why >&2 and exit 1? `>&2` sends the error message to standard error instead of standard output, so it does not mix with the normal output of the script and can be redirected separately (for example `2> err.txt`). `exit 1` ends the script with a non-zero exit code; other programs, and `&&` and `||` in the shell, use it to learn that the script failed. Without `exit 1` the script would go on with a missing file, and the code of the last command would be returned.

CONCLUSION

In this work the bash shell was used as a tool and also studied as a program. Tab completion and Ctrl+R speed up the work; `type`, `which` and PATH show where commands come from and why `cd` has to be a builtin; the exit code 127 marks a command that was not found.

The experiments of Part 2 showed that the shell, not the command, expands wildcards, braces, variables and command substitutions, and that the quoting rules decide which of these steps happen. Part 3 showed that redirections are processed from left to right, that > and >> differ, and how `;`, `&&`, `||` and `&` control the order of commands and jobs.

In Part 4 all eight questions were answered with one-line pipelines of `grep`, `cut`, `sort`, `uniq`, `wc`, `head`, `tail` and `awk`, and every result was checked against another (500 requests in total, 100 failures, 125 requests per page). In Part 5 the best commands became the script `report.sh`, which checks its argument, reports errors on standard error and returns a meaningful exit code.

REFERENCES

[1] Operating Systems, Laboratory Work 3 handout. Lab 3 – The Command Line: Shell, Pipes and Text Filters (Track A). FCIM / FAF / UTM, 2026–2027.

[2] Linux manual pages: bash(1), ls(1), grep(1), cut(1), sort(1), uniq(1), wc(1), awk(1).